



discret v0.6.2

A backend to create peer to peer (P2P) applications, using a GraphQL inspired syntax

#p2p #graphql #quic

Follow

Readme

3 Versions

Dependencies

Dependents

Discret: Create local first, peer to peer application (P2P) using a GraphQL inspired API

Discret hides the complexity of peer to peer networks and reduces it to a data access problem.

The API allows you to:

- manage your data using a GraphQL syntax,
- add access right to your data (in graphql too),
- create and accept invites from other peers.

Discret will synchronize your data with other peers, depending on the access right you have given to those peers.

More details and tutorials are available in the [documentation site](#)

Example

The following example creates a very basic chat application. If you build and run this program on several different folder or local network devices you should be able to chat with yourself.

```
use std::{io, path::PathBuf};
use discret::{
    derive_pass_phrase, zero_uid, Configuration, Discret,
    Parameters, ParametersAdd, ResultParser,
};
use serde::Deserialize;
//the application unique identifier
const APPLICATION_KEY: &str = "github.com/discretlib/rust_example_simple_chat";
#[tokio::main]
async fn main() {
    //define a datamodel
    let model = "chat {
        Message{
            content:String
        }
    }";
    //this struct is used to parse the query result
    #[derive(Deserialize)]
    struct Chat {
        pub id: String,
        pub mdate: i64,
        pub content: String,
    }
```

```

let path: PathBuf = "test_data".into(); //where data is stored
//used to derives all necessary secrets
let key_material: [u8; 32] = derive_pass_phrase("my login", "my password");
//start the discreet application
let app: Discret = Discret::new(
    model,
    APPLICATION_KEY,
    &key_material,
    path,
    Configuration::default(),
)
.await
.unwrap();
//listen for events
let mut events = app.subscribe_for_events().await;
let event_app: Discret = app.clone();
tokio::spawn(async move {
    let mut last_date = 0;
    let mut last_id = zero_uid();
    let private_room: String = event_app.private_room();
    while let Ok(event) = events.recv().await {
        match event {
            //triggered when data is modified
            discret::Event::DataChanged(_) => {
                let mut param = Parameters::new();
                param.add("mdate", last_date).unwrap();
                param.add("id", last_id.clone()).unwrap();
                param.add("room_id", private_room.clone()).unwrap();
                //get the latest data, the result is in the JSON format
                let result: String = event_app
                    .query(
                        "query {
                            res: chat.Message(
                                order_by(mdate asc, id asc),
                                after($mdate, $id),
                                room_id = $room_id
                            ) {
                                id
                                mdate
                                content
                            }
                        }",
                        Some(param),
                    )
                    .await
                    .unwrap();
                let mut query_result = ResultParser::new(&result).unwrap();
                let res: Vec<Chat> = query_result.take_array("res").unwrap();
                for msg in res {
                    last_date = msg.mdate;
                    last_id = msg.id;
                    println!("you said: {}", msg.content);
                }
            }
            _ => {} //ignores other events
        }
    }
});
//data is inserted in your private room
let private_room: String = app.private_room();
let stdin = io::stdin();
let mut line = String::new();
println!("{}", "Write Something!");
loop {
    stdin.read_line(&mut line).unwrap();

```

```

        if line.starts_with("/q") {
            break;
        }
        line.pop();
        let mut params = Parameters::new();
        params.add("message", line.clone()).unwrap();
        params.add("room_id", private_room.clone()).unwrap();
        app.mutate(
            "mutate {
                chat.Message {
                    room_id:$room_id
                    content: $message
                }
            }",
            Some(params),
        )
        .await
        .unwrap();
        line.clear();
    }
}

```

Features

Discret provides a blocking (*DiscretBlocking*) and a non blocking (*Discret*) API.

On local network, peer connection happens without requiring any server. For peer to peer connection over the Internet, a discovery server is needed to allow peers to discover each others. The *discret* lib provides an implementation of the discovery server named *Beacon*.

The library provides strong security features out of the box:

- data is encrypted at rest by using the SQLCipher database
- encrypted communication using the QUIC protocol
- data integrity: each rows is signed with the peer signing key, making it very hard to synchronize bad data
- access control via Rooms

Limitations

As data lives on your devices, *Discret* should only be used for applications with data generated by "real person", with hundreds of peers at most. It is not suited for large scale applications and communities with thousands of peoples.

It currently only support text data but supports for file synchronization is planned.

Connection over the internet is not 100% guaranteed to work, because certain types of enterprise firewalls will block the connection attempts.

Please, be warned that P2P connections leaks your IP address and should only be used with trusted peer. This leak exposes you to the following threats:

- Distributed denial of service (DDOS)
- Leak of your "Real World" location via geolocation services.
- State sponsored surveillance: A state watching the network could determine which peer connect to which, giving a lot of knowledge about your social network.

Platform Support

- Linux: Tested
- Windows: Tested
- macOS: not tested, should work

- Android: works on arch64 architecture. Architectures i686 and x86_64 have some low level linker issues when working with Flutter.
- iOS: not tested

Metadata

📅 8 months ago

🌐 2021 edition

🔧 non-standard

📦 181 KiB

Install

Run the following Cargo command in your project directory:

```
cargo add discret
```

Or add the following line to your Cargo.toml:

```
discret = "0.6.2"
```

Homepage

🔗 discretlib.github.io/doc

Documentation

📖 docs.rs/discret/0.6.2

Owners

 [adsalais](#)

[Report crate](#)

Stats Overview

↓ **2,150**

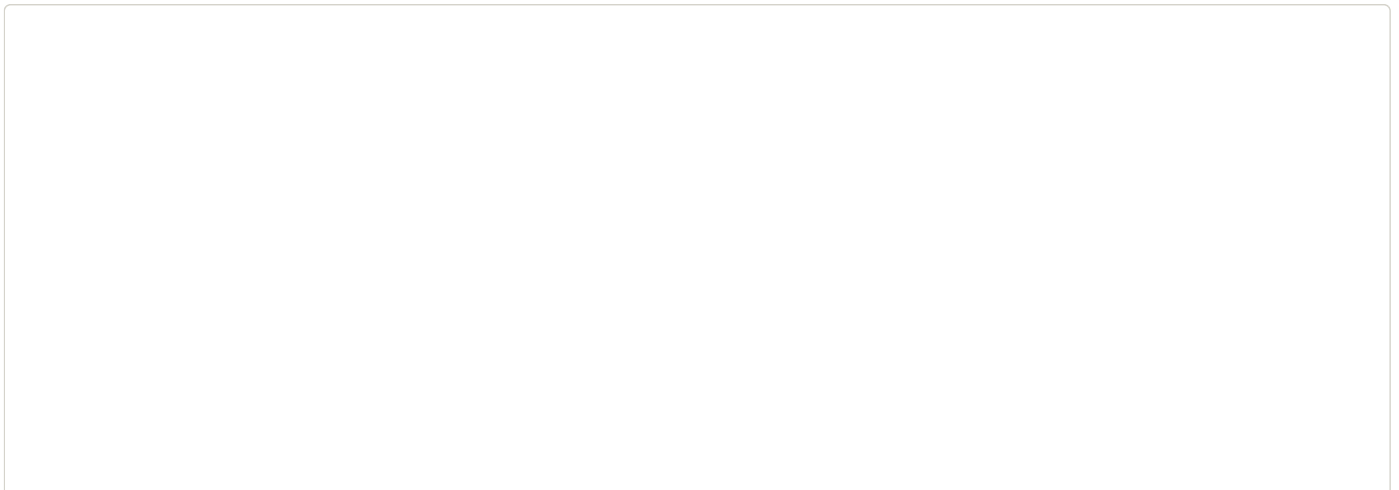
Downloads all time

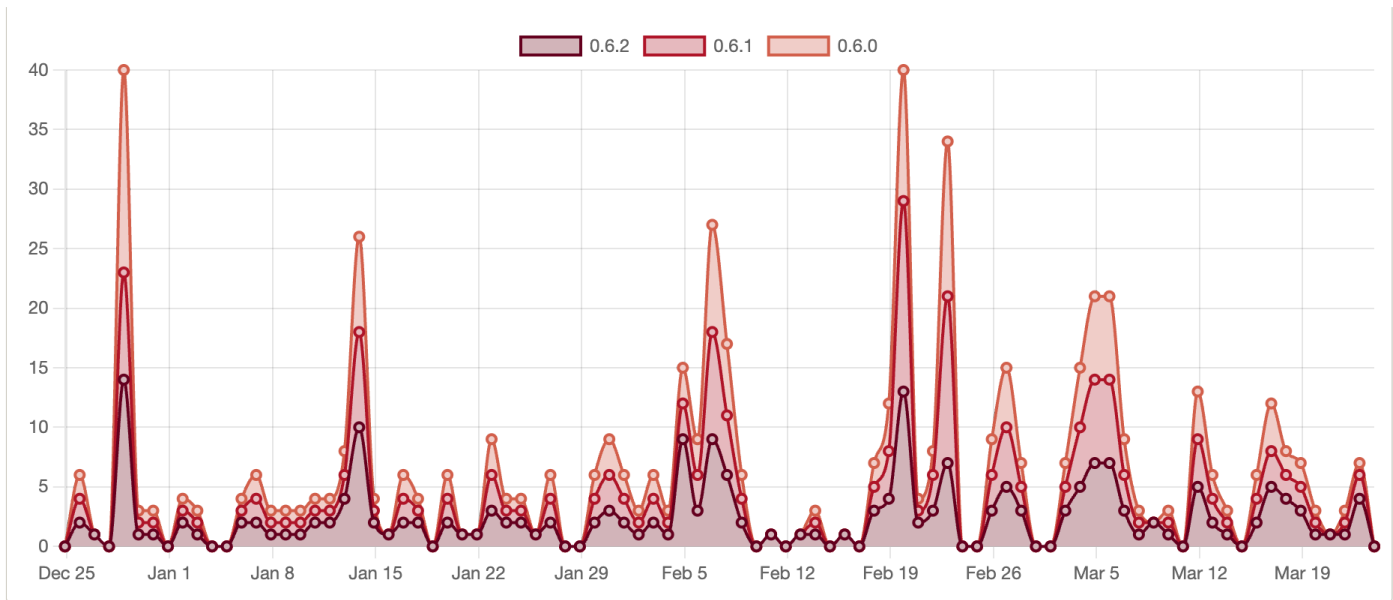
📦 **3**

Versions published

Downloads over the last 90 days

Display as Stacked ▼





Rust

rust-lang.org

[Rust Foundation](#)

[The crates.io team](#)

Get Help

[The Cargo Book](#)

[Support](#)

[System Status](#)

[Report a bug](#)

Policies

[Usage Policy](#)

[Security](#)

[Privacy Policy](#)

[Code of Conduct](#)

[Data Access](#)

Social

[rust-lang/crates.io](https://github.com/rust-lang/crates.io)

[#t-crates-io](https://twitter.com/t-crates-io)

[@cratesiostatus](https://twitter.com/cratesiostatus)